

Topic Outline

- ## Introduction

1. Inter-process Communication (IPC)

Types of IPC:

- Example:**

- Diagram:**

2. Message Passing

Message passing is the **primary mode of IPC in distributed systems.**

A. Synchronous Message Passing

- The sender **waits for the receiver** to acknowledge the message before continuing.

- Blocking communication.
- Sender and receiver must be active simultaneously.
- Easier to implement but slower due to waiting.

Example:

- An online banking system:
 - Client requests fund transfer.
 - Client waits for the server to confirm success before proceeding.

Diagram:

Sender ----Send----> Receiver

Sender <---Ack----- Receiver

B. Asynchronous Message Passing

Definition:

- The sender **sends the message and continues** without waiting for acknowledgment.

Characteristics:

- Non-blocking communication.
- More scalable and suitable for high-latency networks.
- Harder to ensure message order or delivery.

Example:

- Email system or message queues (Kafka, RabbitMQ):
 - User sends an email.
 - Email server queues the message and delivers it later.

Diagram:

Sender ----Send----> Receiver (Queue)

Sender continues processing

Receiver picks message from queue later

3. Remote Procedure Call (RPC)

Definition:

RPC allows a process to **call a procedure on a remote machine as if it were local**.

How RPC Works:

1. **Client Stub:** Acts as a local representative for the remote procedure.
2. **Server Stub:** Receives request, executes procedure, and returns result.
3. **Network Layer:** Transports request and response.

Example:

- Frontend calls getUserDetails(userId) on a remote database server.
- Client stub converts the function call into a network message (marshalling).
- Server stub executes the function and returns data.

Diagram:

Client Process

```

|
| Call getUserDetails()
v

```

Client Stub -----> Network -----> Server Stub

Execute Procedure

Return Result

Challenges of RPC:

- Network failures can interrupt calls.
- Latency higher than local function calls.
- Requires **marshalling** for data exchange.

4. Remote Method Invocation (RMI)

Definition:

- RMI extends RPC to **object-oriented programming**, allowing methods to be called on **remote objects**.
- Common in Java systems.

Key Features:

- Supports objects, inheritance, and polymorphism.
- Uses **serialization** to send objects over the network.

Example:

- Java client invokes `remoteAccount.deposit(100)` on a remote Account object.

Difference between RPC and RMI:

Feature	RPC	RMI
Paradigm	Procedural / function	Object-oriented / method
Language support	Language-agnostic	Typically Java-specific
Data types	Basic types, structures	Objects, classes

5. Serialization, Marshalling, and Network Protocols

A. Serialization

- Converting data (objects, structures) into a **byte stream** to transmit over the network.
- Examples: JSON, XML, Protocol Buffers, Java Serialization.

Example:

- Sending a Java object `User {name: "Alice", age: 25}`
 - Serialized to byte stream: `{ "name": "Alice", "age": 25 }`

B. Marshalling

- Preparing data for transmission, including serialization and adding metadata like type and size.
- Unmarshalling is the reverse process on the receiver side.

Example:

- User object → Serialized to JSON → Add type metadata → Send over TCP → Receiver unmarshals JSON → Reconstruct User object.

C. Network Protocols

- Define **rules for communication** between processes.

Common Protocols:

Protocol	Features	Example Use
TCP	Reliable, connection-oriented	RPC, RMI
UDP	Fast, connectionless, may drop packets	Real-time streaming
HTTP/HTTPS	High-level, request-response	Web APIs, microservices
gRPC	Modern RPC framework using Protocol Buffers	Cloud microservices

Example:

- Microservices communicate via gRPC using TCP for reliability.

6. Summary Table

Concept	Definition / Key Points	Example
IPC	Process communication	Microservices exchange messages
Message Passing	Exchange messages between processes	HTTP requests between services
Synchronous	Sender waits for response	Banking transfer confirmation
Asynchronous	Sender continues immediately	Email, Kafka queues
RPC	Remote procedure call	<code>getUserDetails()</code> on a server

RMI	Remote method invocation	remoteAccount.deposit(100) in Java
Serialization	Convert object to byte stream	JSON, Protocol Buffers
Marshalling	Prepare data for network	Add type and length metadata
Network Protocols	Rules for communication	TCP, UDP, HTTP, gRPC

7. Topic Summary

- **Message passing** is the fundamental communication mechanism in distributed systems.
- **Synchronous vs asynchronous** communication trade-offs: blocking vs performance.
- **RPC and RMI** simplify remote communication by abstracting the network.
- **Serialization and marshalling** are essential to transmit complex data structures.
- Choosing the right **protocol** affects reliability, latency, and scalability.

Topic 3: Processes and Threads

Topic Outline

1. Distributed processes and threads
2. Threading issues in distributed environments
3. Concurrency and parallelism
4. Process migration

1. Processes and Threads

Process

A **process** is a program in execution. It is an active entity that includes:

- Program code
- Program counter
- Stack and heap
- Data section
- Operating system resources (files, I/O devices)

Characteristics

- Each process has its **own address space**
- Processes are **isolated** from each other
- Communication requires **Inter-Process Communication (IPC)** mechanisms
- Context switching is **slow and costly**

Advantages

- Strong fault isolation
- Enhanced security

Disadvantages

- Higher memory usage
- Slower inter-process communication

Thread

A **thread** is the smallest unit of execution within a process.

Characteristics

- Threads share:
 - Code segment
 - Data segment
 - Open files
- Each thread has its own:
 - Program counter
 - Stack
 - Registers

Advantages

- Lightweight
- Faster context switching
- Efficient communication via shared memory
- Improved responsiveness

Disadvantages

- Requires careful synchronization
- A faulty thread can crash the entire process

2. Distributed Processes and Threads

Distributed Processes

A **distributed process** consists of multiple processes executing on different machines but working together as a single logical task.

Features

- No shared memory
- Communication via **message passing**
- Location transparency is desirable
- Independent failures possible

Examples

- Distributed file systems
- Cloud-based microservices
- Distributed databases

Distributed Threads

A **distributed thread** is a logical execution flow that may span across multiple nodes in a distributed system.

Features

- Execution may migrate between machines
- Managed by middleware rather than OS alone

Advantages

- Load balancing
- Improved resource utilization
- Simplified programming abstraction

Challenges

- Network latency
- State management
- Fault tolerance

3. Threading Issues in Distributed Environments

Distributed environments introduce additional complexity compared to single-machine systems.

Major Issues

1. Synchronization

- Shared memory is unavailable
- Synchronization achieved via:
 - Message passing
 - Distributed locks
 - Token-based algorithms
 - Consensus protocols

2. Communication Latency

- Network delays are unpredictable
- Threads must tolerate slow or lost messages

3. Fault Tolerance

- Partial failures are common
- A thread may fail due to node or network failure
- Requires:
 - Replication
 - Checkpointing
 - Recovery mechanisms

4. Scheduling

- Local schedulers lack global knowledge
- Coordinating thread execution across nodes is complex

5. Deadlocks

- Distributed deadlocks are harder to detect
- Requires global state detection algorithms

6. Security

- Threads communicate over insecure networks
- Needs encryption, authentication, and authorization

4. Concurrency and Parallelism

Concurrency

Concurrency refers to the ability of a system to handle multiple tasks at the same time by **interleaving execution**.

Key Points

- Tasks may not run simultaneously
- Improves system responsiveness
- Essential for multitasking systems

Example

- Multiple users accessing a web server

Parallelism

Parallelism refers to executing multiple tasks **simultaneously**.

Key Points

- Requires multiple CPUs or cores
- Improves throughput and performance
- Used in compute-intensive tasks

Example

- Scientific simulations on clusters

Concurrency vs Parallelism

Aspect	Concurrency	Parallelism
Execution	Interleaved	Simultaneous
Focus	Structure	Performance
Hardware	Single or multi-core	Multi-core or distributed

5. Process Migration

Definition

Process migration is the transfer of a running process from one machine to another without stopping the application.

Types of Migration

1. Strong Migration

- Entire execution state is transferred
- Process resumes execution from the exact point
- Complex and resource-intensive

2. Weak Migration

- Only code and data are migrated
- Execution restarts from a predefined checkpoint

Reasons for Process Migration

- Load balancing

- Fault tolerance
- Data locality
- System maintenance

Steps in Process Migration

1. Suspend the process
2. Transfer process state
3. Restart process at destination
4. Resume execution

Challenges

- Transferring open files and network connections
- Security and trust issues
- Platform compatibility
- Performance overhead

Topic Summary

- Processes and threads form the foundation of modern operating and distributed systems.
- When extended across networks, issues such as synchronization, communication latency, and fault tolerance arise.
- Understanding concurrency, parallelism, and process migration is crucial for designing efficient, scalable, and reliable distributed systems.